

Programming Self-Assessment



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Low-Level Synthesis

1 Background

This task aims at enabling students of Low-Level Synthesis to assess whether their programming skills are generally sufficient to solve the Mini Tasks. It deals with the relatively simple problem of transforming the BLIF representation of functions into the positional cube notation (PCN). Since the understanding and handling of internal representations plays a role in almost all Mini Tasks, being able to solve such problems programmatically represents an important competence for the course. However, solving the task is no strict requirement for participation in the course, it only serves as a basis for the self-assessment of one's programming skills (Section 3).

The task may be solved in Java using the provided framework. It contains a simple BLIF parser (in `Main.java`) as well as classes representing functions (`Function.java`) and cubes (`Cube.java`). They contain setter and getter functions and implementations of `toString()`. The cube class provides the method `intersect(Cube other)` which calculates the intersection of a cube with another one. Furthermore, `isLegal()` checks that a cube does not contain the illegal `00`-symbol. Please take a look at the code to get used to the framework. Figure 1 shows an exemplary run configuration for the IDE IntelliJ.

Alternatively, the task may be solved using another programming language. In this case, the basic functionality of the framework has to be implemented from scratch. Please note that an efficient implementation of the PCN, e.g., using Java `BitSet` as done in the framework, is vital for some of the Mini Tasks. Using a string representation, as often done in Python, will lead to performance issues.

2 Tasks

2.1 Task 1: Transform BLIF into PCN

Complete the method `addLine(String line)` in `Function.java` to transform lines of the BLIF specification into a cube in PCN. Both representations have been introduced in the lecture. The cube class inherits from the Java `BitSet`¹ class.

2.2 Task 2: Intersect Functions

Complete the main method of `Main.java` to intersect all parsed functions. They are stored in the list `parsedFunctions`. Use the given list `intersectedFunctions` to store the results of the intersection. To intersect two functions, each cube of the first function has to be intersected with each cube of the second function. Keep in mind that the intersection does not contain illegal cubes.

¹<https://docs.oracle.com/javase/8/docs/api/java/util/BitSet.html>

3 Self-Assessment

If you can solve the task in one or two days, you should be well prepared for working on the much more complex Mini Tasks. If you need less than 2 hours, you are very quick. **If you cannot solve the task, or need longer than two days, you may run into problems and should reevaluate if you further participate in the course.**

4 Verification of your Implementation

Process the given BLIF file by providing `resources/example.blif` as the argument for the program (see figure 1).

If you have correctly implemented Task 1, the program will output the parsed functions in the following way:

```
===== TASK 1 =====
```

Parsed functions:

```
a b c d e f -> o1
111111010101
111101111111
111011111110
101111101001
```

```
a b c d e f -> o2
111110010101
111011011101
111010110101
101010111101
```

```
==> CORRECT
```

If you have implemented Task 2 correctly, the program will further output the intersected functions in the following way:

```
===== TASK 2 =====
```

Intersected functions:

```
a b c d e f -> o1
111110010101
111011010101
111010010101
101010010101
111001011101
101010101001
```

```
a b c d e f -> o2
111110010101
111011010101
111001011101
111010010101
101010010101
101010101001
```

```
==> CORRECT
```

This output can also be found in the file `golden-output/example.txt`.

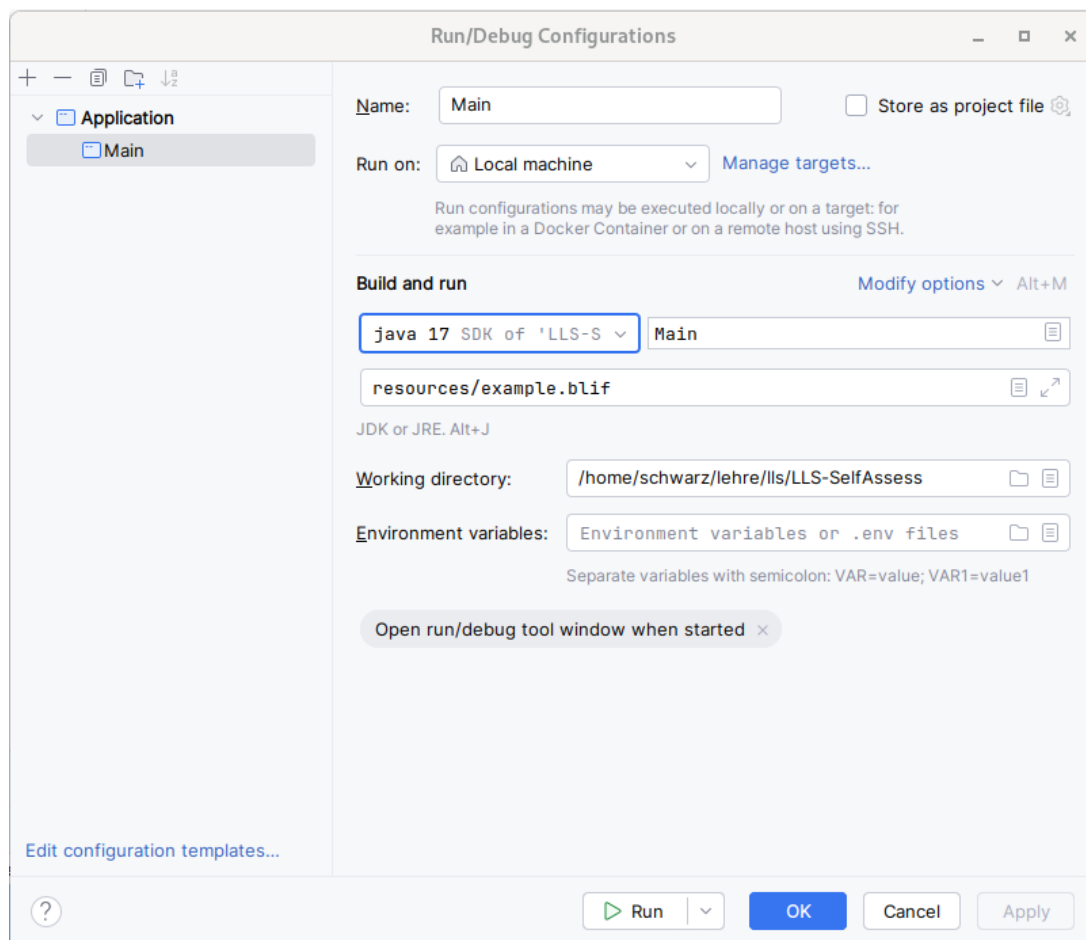


Figure 1: Exemplary run configuration. The program takes one argument, the BLIF file to be processed.